

wProject Executable Files

Date	5 july 2026
Team ID	
Project Name	A Comprehensive Measure of Well-Being
Maximum Marks	3 Marks

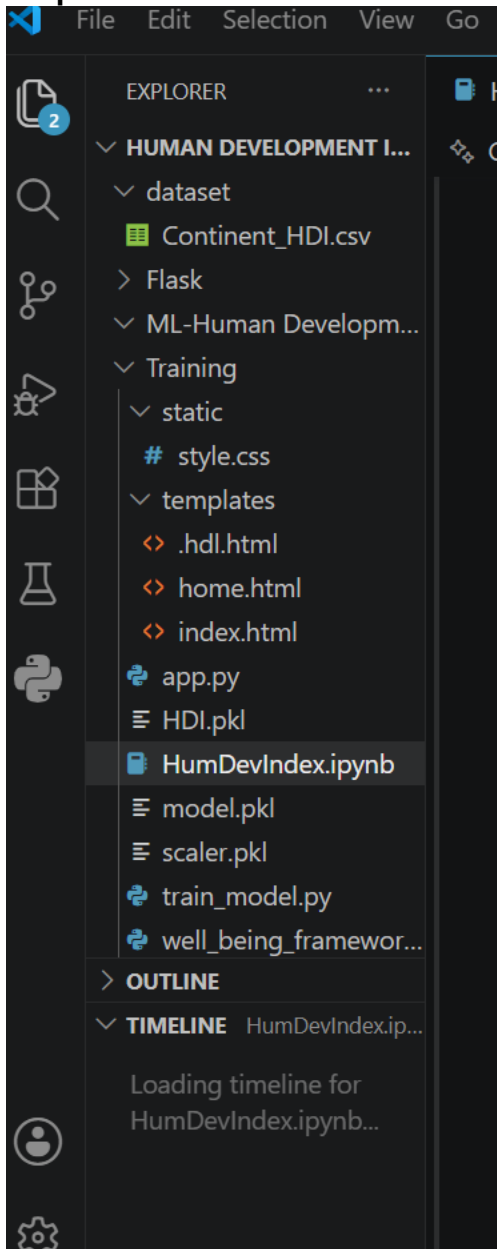
Project Executable Files

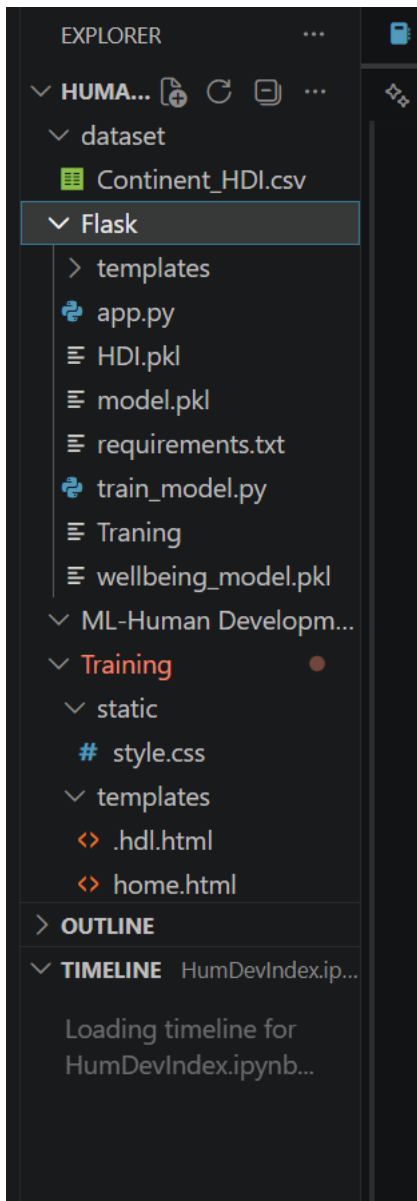
This section requires submission of all executable and deployable files for your project. Ensure all files are properly organized, named, and uploaded to the specified submission platform. The evaluator must be able to run or deploy your solution using the provided files and instructions.

Step 1: Submission Checklist

S.No	Item to Submit	Submitted (Yes / No / NA)
1	Complete source code (all files and folders)	Yes
2	README / Setup Guide (instructions to run the project)	Yes
3	requirements.txt / package.json / dependency file	Yes
4	Database schema / seed files (if applicable)	NA (Utilizes serialized .pkl models)
5	Environment configuration file (.env.example or similar)	NA
6	Deployed application URL (if hosted)	Yes
7	APK / Executable binary (if applicable for mobile/desktop apps)	NA
8	Dockerfile / Containerization config (if applicable)	NA
9	Test files and test results	Yes
10	Demo video or walkthrough (if required)	Yes

Step 2: File / Folder Structure





Step 3: Deployment / Access Details

Field	Details
Hosted / Deployed URL	https://hdiproject.onrender.com
Login Credentials (Demo)	NA - The prediction dashboard is publicly accessible for evaluation.
Platform / Hosting Provider	Render Cloud Platform
Repository Link	https://github.com/gaddamaiswarya1234-beep/A-Comprehensive-Measure-of-Well-Being

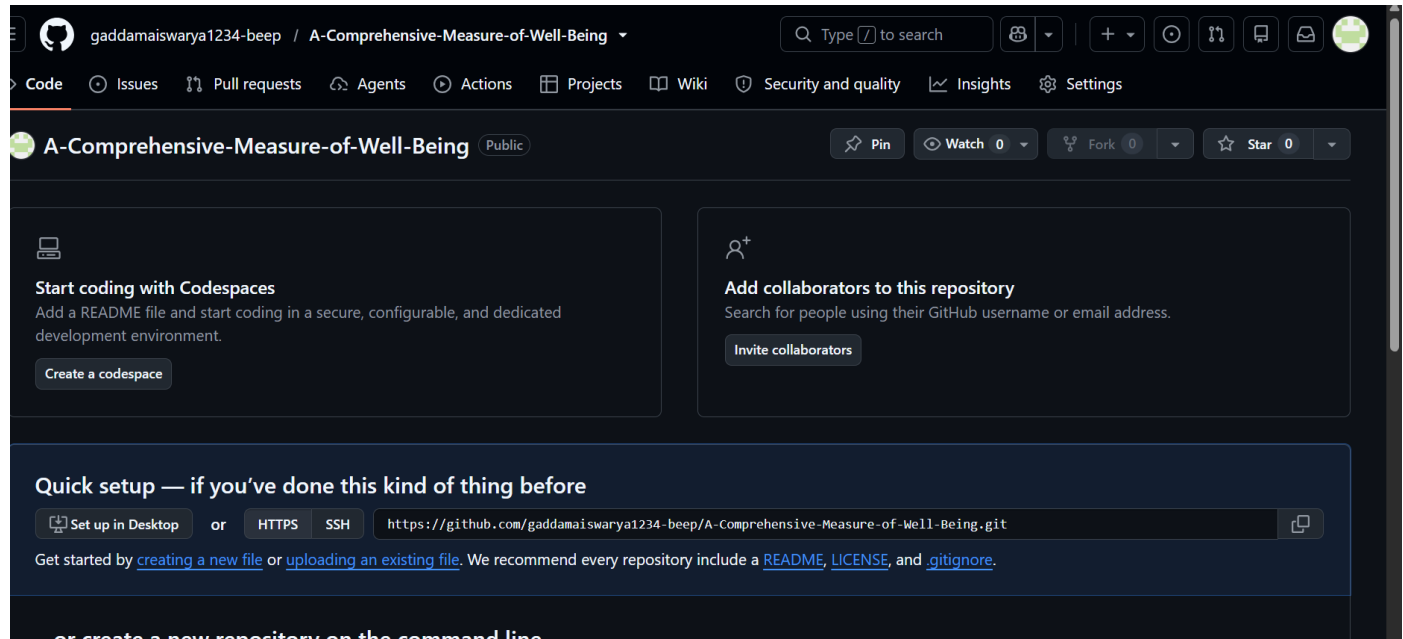
Demo Video Link	https://1drv.ms/v/c/5798EF39647C131B/IQD9MiBRMIbST5VhZV8LYSOFAVOpEQI7RekzDAT0bTXvmCI?e=YIPMKD
------------------------	---

Step 4: Run Instructions

Provide clear, step-by-step instructions for the evaluator to run your project locally:

Pre-requisites: Python 3.8 or higher, Git, and a stable internet connection.

Step 1: Clone the repository to your local machine: `git clone [Insert your GitHub URL]`



or create a new repository on the command line

Step 2: Navigate into the project root directory using your terminal.

```
PS C:\Users\DELL\OneDrive\Desktop\Human Development Index> cd Flask
PS C:\Users\DELL\OneDrive\Desktop\Human Development Index\Flask> python app.py
* Serving Flask app 'app'
* Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on http://127.0.0.1:5000
Press CTRL+C to quit
* Restarting with stat
* Debugger is active!
* Debugger PIN: 837-371-456
```

Step 3: Install all required Python dependencies: `pip install -r requirements.txt`

```
import numpy as np
import pandas as pd

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score

import matplotlib.pyplot as plt
import seaborn as sns

from flask import Flask, render_template, request

pd.set_option('display.max_columns', None)
pd.set_option('display.width', 1000)

print("=====")
print("A Comprehensive Measure of Well-Being")
print("All required libraries imported successfully!")
print("=====")
```

```
=====
A Comprehensive Measure of Well-Being
All required libraries imported successfully!
=====
```

Step 4: Start the local Flask development server: `python app.py`

EXPLORER

...

...

▼ HUMA...

dataset

Continent_HDI.csv

> Flask

▼ ML-Human Developm...

▼ Training

static

style.css

templates

hd.html

home.html

index.html

app.py

HDI.pkl

HumDevIndex.ipynb

model.pkl

scaler.pkl

train_model.py

well_being_fra... 9+

> OUTLINE

▼ TIMELINE app.py

File Saved 21 hrs

File Saved

File Saved 23 hrs

File Saved 1 day

File Saved

HumDevIndex.ipynb

result.html

index.html Flask\...

hd.html

app.py Training

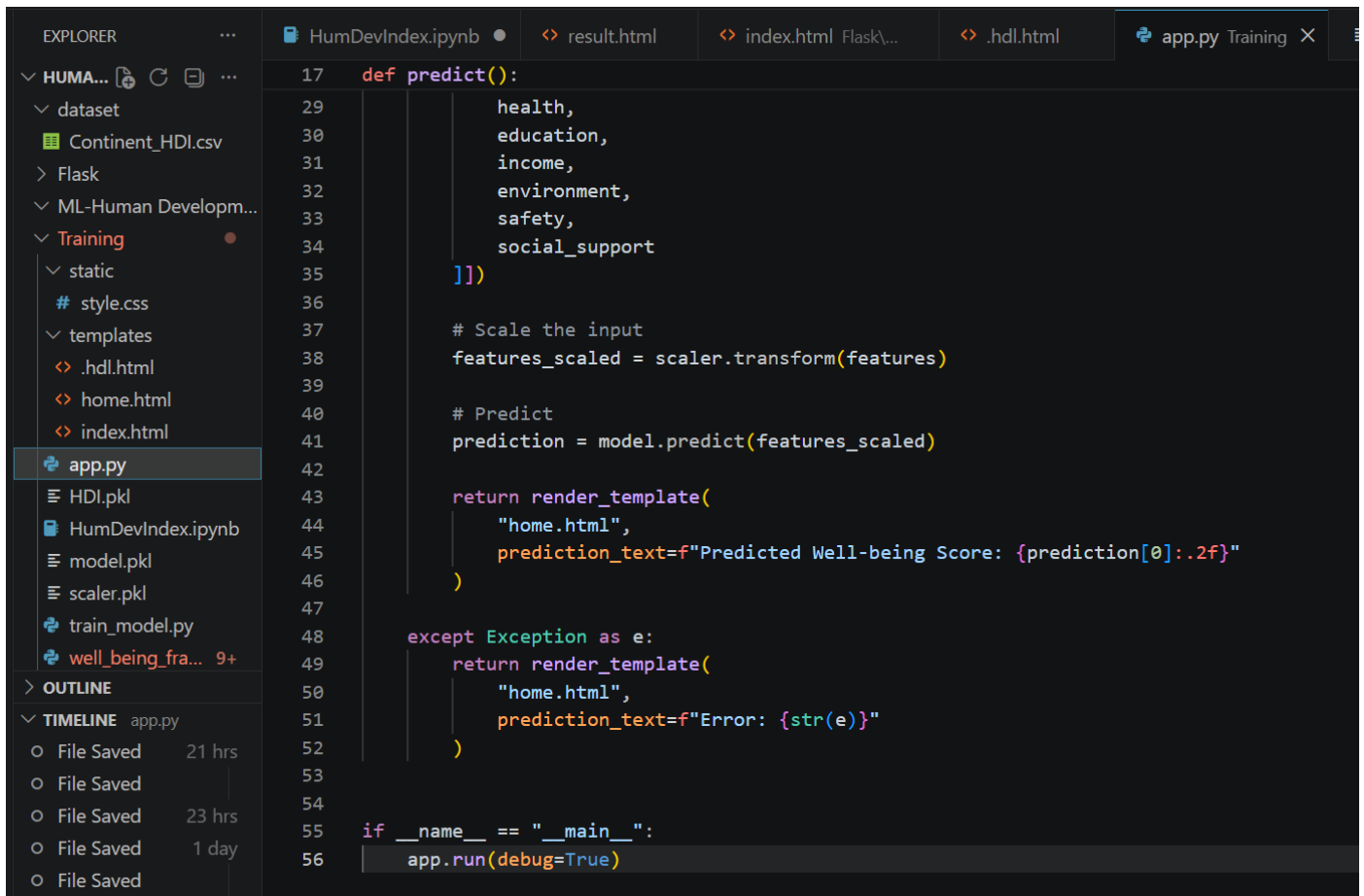
Trani

```

1  from flask import Flask, render_template, request
2  import pickle
3  import numpy as np
4
5  app = Flask(__name__)
6
7  # Load the trained model and scaler
8  model = pickle.load(open("model.pkl", "rb"))
9  scaler = pickle.load(open("scaler.pkl", "rb"))
10
11 @app.route("/")
12 def home():
13     return render_template("home.html")
14
15
16 @app.route("/predict", methods=["POST"])
17 def predict():
18     try:
19         # Read input values
20         health = float(request.form["health"])
21         education = float(request.form["education"])
22         income = float(request.form["income"])
23         environment = float(request.form["environment"])
24         safety = float(request.form["safety"])
25         social_support = float(request.form["social_support"])
26
27         # Create input array
28         features = np.array([[
29             health,
30             education,
31             income

```

Ln 56, Col 24 Spaces: 4 UTF-8 CRLF (1) Py



The screenshot shows a JupyterLab environment with a file explorer on the left and a code editor on the right. The file explorer shows a project structure with folders like 'dataset', 'Flask', 'ML-Human Developm...', and 'Training'. The 'Training' folder is expanded, showing files like 'static', 'style.css', 'templates', '.hdl.html', 'home.html', 'index.html', and 'app.py'. The 'app.py' file is selected. The code editor shows the following Python code:

```

17 def predict():
29     health,
30     education,
31     income,
32     environment,
33     safety,
34     social_support
35
36
37
38     # Scale the input
39     features_scaled = scaler.transform(features)
40
41     # Predict
42     prediction = model.predict(features_scaled)
43
44     return render_template(
45         "home.html",
46         prediction_text=f"Predicted Well-being Score: {prediction[0]:.2f}"
47     )
48
49 except Exception as e:
50     return render_template(
51         "home.html",
52         prediction_text=f"Error: {str(e)}"
53     )
54
55 if __name__ == "__main__":
56     app.run(debug=True)
  
```

Step 5: Open your web browser and navigate to <http://127.0.0.1:5000> to access the application.

Comprehensive Measure of Well-Being

Predicted Well-Being Score: 72.32

Predict Again

Step 5: Known Issues / Limitations

S.No	Known Issue / Limitation	Workaround / Status
1	Server "Cold Start" Latency	Render free tier spins down inactive instances. The first request takes ~3-5 seconds to wake the server. Subsequent requests are instant.
2	UI scaling on extremely small screens	The data input grid may stack tightly on screens under 360px wide. Advised to view on standard mobile or desktop dimensions.